

Implementación de técnicas de Deception en un Honeypot Web con Wazuh

Marta Bargués Bosch



1. Introducción.....	3
1.1 Contexto del proyecto.....	3
1.2 Objetivos.....	3
1.3 Punto de partida.....	3
1.4 Enfoque del proyecto.....	4
2. Diseño de los señuelos.....	5
2.1 Archivo .env falso.....	5
2.2 Panel de administración falso.....	6
2.3 Falsa webshell de WordPress.....	6
3. Implementación de los señuelos.....	7
3.1 Creación del archivo .env falso.....	7
3.2 Configuración de la ruta canario en Nginx.....	7
3.3 Creación del panel de administración falso.....	8
3.4 Registro de los intentos de acceso.....	9
3.5 Ejecución automática y aislamiento del panel.....	9
3.6 Pruebas de funcionamiento.....	10
3.6.1 Acceso al archivo .env.....	10
3.6.2 Prueba de la ruta canario.....	11
3.7 Tercer señuelo: Falsa webshell.....	11
3.7.1 Análisis del tráfico web para elegir el tercer señuelo.....	11
3.7.2 Identificación de la ruta /wp-content/plugins/hellopress/wp_filemanager.php...	12
3.7.3 Investigación de la ruta detectada.....	12
3.7.4 Creación de la falsa webshell.....	13
4. Integración y detección con Wazuh.....	14
4.1 Monitorización de los logs de Nginx.....	14
4.2 Reglas personalizadas de detección.....	14
4.3 Prueba de las reglas.....	15
5. Análisis de la actividad detectada.....	16
5.1 Análisis de las rutas más buscadas.....	17
5.2 Análisis de una IP con alta actividad.....	17
5.3 Adaptación del señuelo al tráfico real.....	18
5.4 Resultados observados.....	19
5.5 Detección de actividad compatible con AndroXGh0st.....	20
5.6 Limitaciones observadas.....	21
6. Conclusiones.....	21

1. Introducción

1.1 Contexto del proyecto

Este proyecto nace como continuación del honeypot que se había creado anteriormente en AWS. En aquella primera parte se dejó un servidor expuesto a Internet para observar qué tipo de actividad recibía, principalmente conexiones SSH y peticiones contra la web. Después de ver todo lo que llegaba al honeypot surgió la idea de ir un poco más allá. En lugar de limitarnos a registrar los accesos, se decidió añadir distintos señuelos para comprobar qué estaban buscando los bots y qué hacían cuando encontraban algo que podía parecer interesante.

A partir de ahí se fueron creando diferentes señuelos y utilizando Wazuh para recoger y analizar la actividad. Como se ha trabajado con tráfico real de Internet, el proyecto también ha ido evolucionando según lo que se encontraba, adaptando los señuelos al comportamiento observado.

1.2 Objetivos

El objetivo principal del proyecto es ampliar el honeypot creado anteriormente y hacerlo más interactivo. La idea es no limitarse a detectar conexiones o escaneos, sino observar qué buscan los sistemas que llegan al servidor y cómo reaccionan ante diferentes señuelos.

Para esto se plantean los siguientes objetivos:

- Crear distintos señuelos web que puedan llamar la atención de bots y escáneres.
- Registrar los accesos y las interacciones con estos recursos.
- Utilizar Wazuh para centralizar y analizar la actividad recibida.
- Crear reglas propias para identificar fácilmente los accesos a cada señuelo.
- Generar alertas cuando se produzcan interacciones relevantes.
- Analizar el tráfico real recibido desde Internet y los comportamientos observados.
- Adaptar los señuelos según lo que se vaya descubriendo durante las pruebas.

1.3 Punto de partida

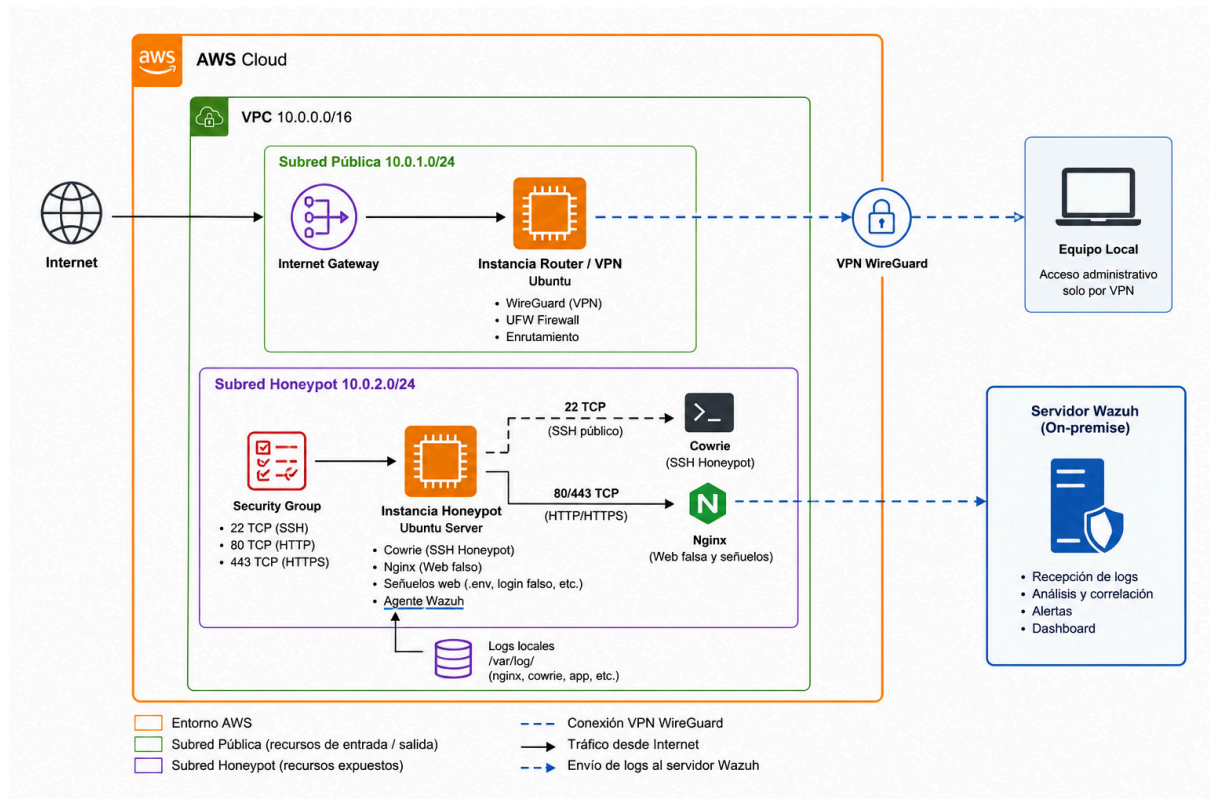
Para este proyecto se parte del honeypot creado anteriormente en AWS, que ya estaba preparado y expuesto de forma controlada a Internet.

La infraestructura contaba con una VPC dividida en varias subredes, una VPN con WireGuard para realizar la administración de forma segura y una instancia Ubuntu donde se

encontraba el honeypot. Las conexiones SSH públicas eran recibidas por Cowrie, mientras que el acceso SSH real al servidor solo podía realizarse a través de la VPN.

También se había instalado Nginx para publicar la web ficticia de Aegis Bank y Wazuh para recoger y analizar la actividad registrada en el honeypot desde el laboratorio local.

Todo este entorno se mantuvo como punto de partida, permitiendo centrar este segundo proyecto directamente en la creación de nuevos señuelos y el análisis de las interacciones que se recibieran.



1.4 Enfoque del proyecto

El proyecto se ha ido desarrollando de forma práctica y poco a poco. En lugar de decidir todos los señuelos desde el principio, se dejó el honeypot expuesto a Internet para ver primero qué estaban buscando realmente los sistemas que llegaban al servidor.

A partir de ahí se fueron creando y ajustando los señuelos. Durante las pruebas aparecieron búsquedas de archivos .env, rutas de WordPress, archivos PHP y posibles webshells, y parte de esta actividad sirvió para decidir qué podía ser interesante simular.

Así el honeypot ha ido evolucionando según la actividad recibida. Esto ha permitido no solo crear los señuelos, sino también observar los resultados, analizarlos con Wazuh y adaptar el entorno a medida que aparecían nuevos comportamientos.

2. Diseño de los señuelos

Para ampliar el honeypot se decidió añadir varios señuelos web que simulan recursos que podrían llamar la atención de un atacante.

La idea es observar qué ocurre cuando un bot o atacante encuentra alguno de estos recursos y decide acceder o interactuar con él.

Todos los señuelos contienen información completamente ficticia y están separados de los servicios reales de administración del servidor, por lo que no proporcionan acceso a ningún recurso sensible.

2.1 Archivo .env falso

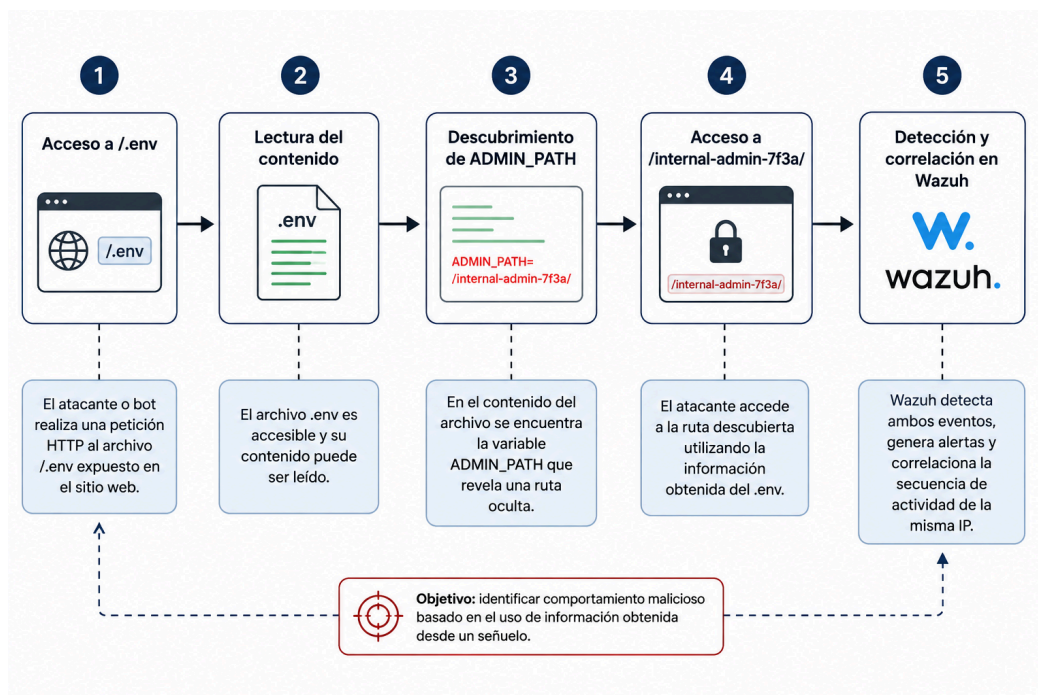
El primer señuelo consiste en publicar a propósito un archivo .env falso. Este tipo de archivos puede resultar atractivo para sistemas automatizados porque normalmente se utilizan para guardar variables de configuración de una aplicación, además ya se observó en el proyecto anterior su búsqueda.

En nuestro caso, el archivo contiene únicamente datos ficticios, como una supuesta base de datos, usuario y contraseña. También incluye una ruta de administración oculta:

```
ADMIN_PATH=/internal-admin-7f3a/
```

Esta ruta no aparece enlazada en ninguna otra parte de la web. De esta forma, si una misma IP solicita primero el archivo .env y posteriormente accede a esta dirección, podemos considerar que probablemente ha utilizado la información encontrada en el señuelo.

El comportamiento que queremos detectar es:



2.2 Panel de administración falso

El segundo señuelo consiste en un panel de administración falso, situado en la ruta que aparece dentro del archivo `.env`.

El panel simula el acceso interno de Aegis Bank mediante un formulario de usuario y contraseña. No existen credenciales válidas, por lo que cualquier intento de acceso siempre será rechazado.

El objetivo es comprobar si alguien que descubre la ruta intenta iniciar sesión y qué nombres de usuario utiliza. Por seguridad, las contraseñas introducidas no se guardan; únicamente se registra la IP de origen, el usuario probado, la fecha y el resultado del intento. La aplicación está desarrollada con Flask y solo es accesible internamente. Nginx actúa como intermediario, evitando que Flask quede expuesto directamente a Internet.

2.3 Falsa webshell de WordPress

El tercer señuelo se diseñó a partir de la propia actividad registrada por el honeypot. En lugar de elegir una nueva ruta al azar, se analizaron en Wazuh las URL que estaban siendo buscadas por diferentes direcciones IP.

Durante este análisis apareció repetidamente la siguiente ruta

`/wp-content/plugins/hellopress/wp_filemanager.php`

Al revisar los registros de Nginx se comprobó que varias IP diferentes habían intentado acceder a este mismo recurso. La ruta parecía estar relacionada con la búsqueda de gestores de archivos o webshells en instalaciones WordPress ya comprometidas, por lo que resultaba interesante utilizarla como señuelo.

Se decidió entonces simular la existencia del archivo haciendo que el servidor respondiera con 200 OK. No se instaló WordPress ni ninguna webshell real. Solo la ruta muestra una página HTML falsa que simula un pequeño gestor de archivos, sin permitir realizar ninguna acción sobre el servidor.

El objetivo era observar qué ocurría cuando uno de los escáneres que ya busca esta ruta volviera a encontrarla y, esta vez, recibiera una respuesta positiva. Así podríamos comprobar si simplemente abandonaba la conexión o si continuaba interactuando con el recurso.

3. Implementación de los señuelos

3.1 Creación del archivo `.env` falso

El primer señuelo consiste en un archivo `.env` falso situado en la raíz de la web.

`/var/www/html/.env`

Este tipo de archivos puede contener información sensible de configuración, por lo que es habitual que bots y escáneres intenten localizarlos.

Para el honeypot se creó uno accesible desde Internet, pero con datos completamente ficticios

```
GNU nano 7.2 /var/www/html/.env
APP_ENV=production
APP_DEBUG=false

DB_HOST=10.0.3.15
DB_DATABASE=aegis_prod
DB_USERNAME=aegis_admin
DB_PASSWORD=R7x9K2mP4vQ8

ADMIN_PATH=/internal-admin-7f3a/
```

Ninguno de estos datos corresponde a usuarios, contraseñas, servidores o servicios reales de la infraestructura.

La parte más importante es la variable `ADMIN_PATH=/internal-admin-7f3a/` que funciona como ruta canario. Esta dirección no aparece en ningún lugar de la web pública, por lo que un visitante normal no debería conocerla.

Si una IP accede primero al `.env` y después intenta visitar `/internal-admin-7f3a/`, podremos relacionar ambos eventos y ver que la información encontrada en el señuelo ha sido utilizada.

3.2 Configuración de la ruta canario en Nginx

Una vez creada la ruta canario, se configuró Nginx para gestionar las peticiones recibidas en `/internal-admin-7f3a/`.

Esta ruta se configuró como proxy inverso hacia la aplicación Flask que mostrará el panel de administración falso

```
ubuntu@honeypot-aegis:~$ sudo sed -n '/location \/internal-admin-7f3a\/, +6p' /etc/nginx/sites-enabled/aegisbank
location /internal-admin-7f3a/ {
    proxy_pass http://127.0.0.1:5000/internal-admin-7f3a/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}
```

De esta forma, Nginx recibe las conexiones desde Internet y las envía internamente a Flask, que únicamente escucha en `127.0.0.1:5000`.

```
ubuntu@honeypot-aegis:~$ sudo ss -lntp | grep 5000
LISTEN 0      128          127.0.0.1:5000      0.0.0.0:*    users:((("python",pid=534,fd=3))
ubuntu@honeypot-aegis:~$
```

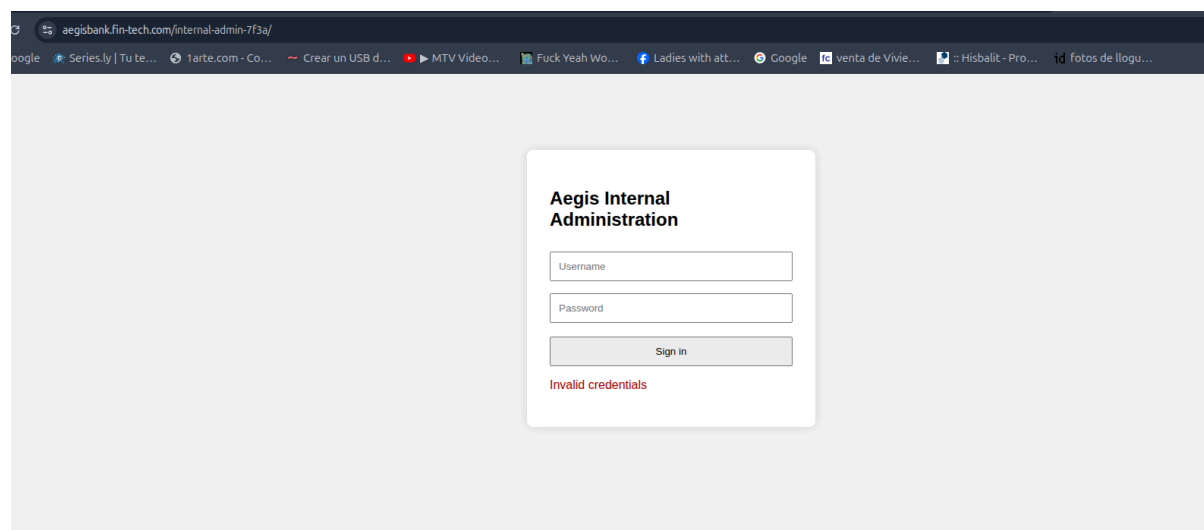
3.3 Creación del panel de administración falso

Una vez configurada la ruta canario, se creó un panel de administración ficticio para simular una zona interna de Aegis Bank.

Para ello se desarrolló una pequeña aplicación con Flask, ubicada en /opt/aegis-login.

```
GNU nano 7.2 /opt/aegis-login/app.py
    margin: 8px 0;
    box-sizing: border-box;
  }
  button {
    width: 100%;
    padding: 10px;
    margin-top: 10px;
  }
  .error { color: #a00; }
</style>
</head>
<body>
<div class="login-box">
  <h2>Aegis Internal Administration</h2>
  <form method="POST">
    <input name="username" placeholder="Username" required>
    <input name="password" type="password" placeholder="Password" required>
    <button type="submit">Sign in</button>
  </form>
  {% if error %}
    <p class="error">Invalid credentials</p>
  {% endif %}
</div>
</body>
</html>
"""
```

El panel muestra un formulario de usuario y contraseña y únicamente responde en la ruta /internal-admin-7f3a/.



El objetivo no es simular una aplicación completa, sino observar si alguien que ha descubierto la ruta intenta utilizarla y qué nombres de usuario prueba.

3.4 Registro de los intentos de acceso

Para poder analizar la actividad del panel falso, Flask registra cada intento de autenticación en `/var/log/aegis-login.log`.

Por cada intento se almacena la fecha, la IP de origen, el usuario probado y el resultado. Gracias a las cabeceras enviadas por Nginx, Flask puede conservar la IP real del cliente que realiza el intento.

```
ubuntu@honeypot-aegis:~$ sudo tail -5 /var/log/aegis-login.log
2026-08-21T14:03:26.548635+00:00 src_ip=79.152.205.50 username=admin result=failed
ubuntu@honeypot-aegis:~$
```

Por seguridad, las contraseñas no se almacenan y todos los intentos de acceso terminan siempre con un resultado fallido.

3.5 Ejecución automática y aislamiento del panel

Para evitar tener que iniciar Flask manualmente cada vez que se reinicia el servidor, se creó el servicio `aegis-login.service`.

El servicio ejecuta automáticamente la aplicación de `/opt/aegis-login` y está configurado para reiniciarse si se detiene.

```
ubuntu@honeypot-aegis:~$ sudo cat /etc/systemd/system/aegis-login.service
[Unit]
Description=Aegis Bank Fake Login Honeypot
After=network.target

[Service]
User=ubuntu
Group=ubuntu
WorkingDirectory=/opt/aegis-login
ExecStart=/opt/aegis-login/venv/bin/python /opt/aegis-login/app.py
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target
```

Después de crearlo, se habilitó y se puso en marcha, al comprobar su estado, el servicio aparecía como `active (running)`.

```

ubuntu@honeypot-aegis:~$ sudo systemctl daemon-reload
ubuntu@honeypot-aegis:~$ sudo systemctl enable aegis-login.service
Created symlink /etc/systemd/system/multi-user.target.wants/aegis-login.service → /etc/systemd/system/aegis-login.service.
ubuntu@honeypot-aegis:~$ sudo systemctl start aegis-login.service
ubuntu@honeypot-aegis:~$ sudo systemctl status aegis-login.service
● aegis-login.service - Aegis Bank Fake Login Honeypot
   Loaded: loaded (/etc/systemd/system/aegis-login.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-08-21 14:01:29 UTC; 8s ago
     Main PID: 2449 (python)
       Tasks: 1 (limit: 627)
      Memory: 20.4M (peak: 22.9M)
         CPU: 243ms
    CGroup: /system.slice/aegis-login.service
            └─2449 /opt/aegis-login/venv/bin/python /opt/aegis-login/app.py

Aug 21 14:01:29 honeypot-aegis systemd[1]: Started aegis-login.service - Aegis Bank Fake Login Honeypot.
Aug 21 14:01:29 honeypot-aegis python[2449]: * Serving Flask app 'app'
Aug 21 14:01:29 honeypot-aegis python[2449]: * Debug mode: off
Aug 21 14:01:29 honeypot-aegis python[2449]: WARNING: This is a development server. Do not use it in a production deployment. Use a production
Aug 21 14:01:29 honeypot-aegis python[2449]: * Running on http://127.0.0.1:5000
Aug 21 14:01:29 honeypot-aegis python[2449]: Press CTRL+C to quit

```

3.6 Pruebas de funcionamiento

Una vez configurados los señuelos, se realizaron varias pruebas desde un equipo externo para comprobar que todo funcionaba bien antes de integrarlo con Wazuh.

3.6.1 Acceso al archivo .env

Primero se comprobó que el archivo falso era accesible desde Internet.

```

mtk@mtk-IdeaPad-3-15ITL6:~$ curl -i https://aegisbank.fin-tech.com/.env
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Date: Mon, 24 Aug 2026 08:46:30 GMT
Content-Type: application/octet-stream
Content-Length: 160
Last-Modified: Tue, 18 Aug 2026 14:59:44 GMT
Connection: keep-alive
ETag: "6a847360-a0"
Accept-Ranges: bytes

APP_ENV=production
APP_DEBUG=false

DB_HOST=10.0.3.15
DB_DATABASE=aegis_prod
DB_USERNAME=aegis_admin
DB_PASSWORD=R7x9K2mP4vQ8

ADMIN_PATH=/internal-admin-7f3a/

```

También se comprobó que la petición había quedado registrada en `/var/log/nginx/access.log`.

```

ubuntu@honeypot-aegis:~$ sudo cat /var/log/nginx/access.log
162.62.231.139 - - [24/Aug/2026:08:29:03 +0000] "GET / HTTP/1.1" 404 134 "-" "Mozilla/5.0 (iPhone; CPU iPhone OS 13_2_3 like
Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/13.0.3 Mobile/15E148 Safari/604.1"
79.152.205.50 - - [24/Aug/2026:08:46:30 +0000] "GET /.env HTTP/1.1" 200 160 "-" "curl/8.5.0"
40.124.186.155 - - [24/Aug/2026:08:58:33 +0000] "GET /actuator/health HTTP/1.1" 404 134 "-" "Mozilla/5.0 zgrab/0.x"
43.157.98.187 - - [24/Aug/2026:09:25:33 +0000] "GET / HTTP/1.1" 400 264 "-" "Mozilla/5.0 (iPhone; CPU iPhone OS 13_2_3 like M
ac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/13.0.3 Mobile/15E148 Safari/604.1"
43.164.196.244 - - [24/Aug/2026:09:28:02 +0000] "GET / HTTP/1.1" 200 1637 "-" "Mozilla/5.0 (iPhone; CPU iPhone OS 13_2_3 like
Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/13.0.3 Mobile/15E148 Safari/604.1"
43.161.217.205 - - [24/Aug/2026:09:39:19 +0000] "GET / HTTP/1.1" 200 1637 "-" "Mozilla/5.0 (iPhone; CPU iPhone OS 13_2_3 like
Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/13.0.3 Mobile/15E148 Safari/604.1"
79.152.205.50 - - [24/Aug/2026:09:48:17 +0000] "GET /.env HTTP/1.1" 200 160 "-" "curl/8.5.0"
47.89.129.126 - - [24/Aug/2026:09:50:36 +0000] "GET / HTTP/1.1" 200 1637 "-" "Mozilla/5.0 (iPhone; CPU iPhone OS 13_2_3 like
Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/13.0.3 Mobile/15E148 Safari/604.1"

```

También observamos que ya existía tráfico real desde internet que no éramos nosotros.

3.6.2 Prueba de la ruta canario

Después se simuló lo que ocurriría si alguien encontrara el .env, leyera su contenido y utilizara la ruta descubierta.

En los logs quedaron registradas las peticiones de las IP.

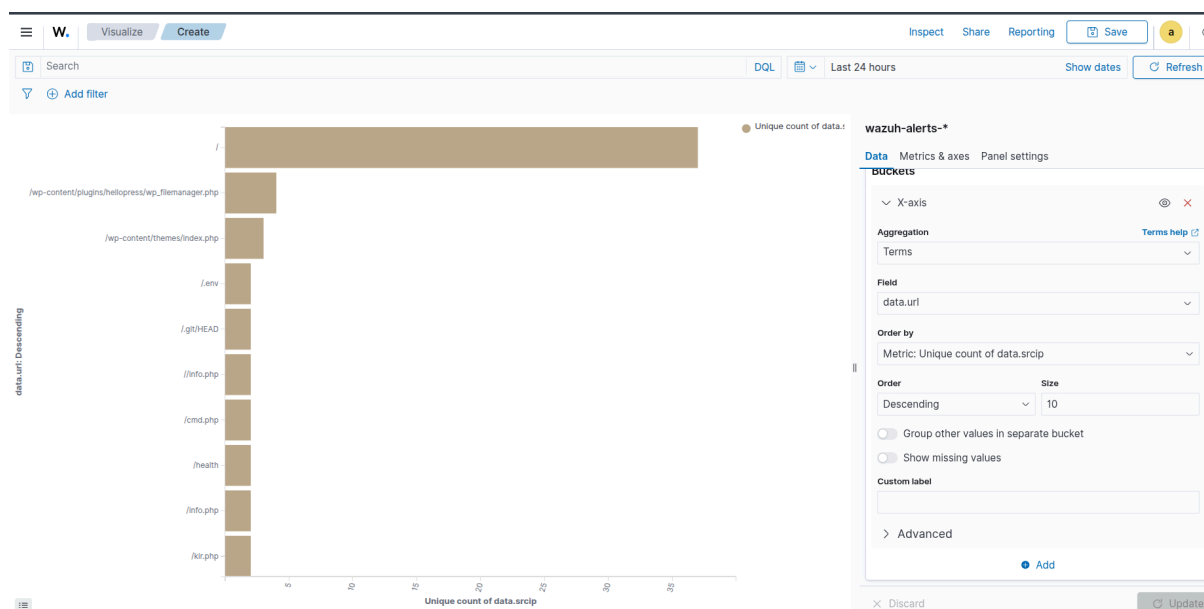
3.7 Tercer señuelo: Falsa webshell

3.7.1 Análisis del tráfico web para elegir el tercer señuelo

Antes de crear el tercer señuelo se decidió revisar la actividad que ya estaba recibiendo el honeypot. En lugar de inventar otra ruta, parecía más interesante ver qué estaban buscando realmente los escáneres que llegaban al servidor y utilizar esa información para decidir el siguiente señuelo.

Para verlo más fácilmente se creó una visualización en Wazuh con las rutas más buscadas. En este caso no se tuvo en cuenta sólo el número de peticiones, sino cuántas IP diferentes habían intentado acceder a cada ruta.

Así se evitaba que una sola IP, haciendo muchas peticiones seguidas, alterara los resultados y se podía ver mejor qué recursos estaban siendo buscados.



3.7.2 Identificación de la ruta /wp-content/plugins/hellopress/wp_filemanager.php

Al revisar los resultados aparecían muchas peticiones bastante habituales, como accesos a /, pero también algunas rutas mucho más concretas. Aparte de /env una de las que más llamó la atención fue /wp-content/plugins/hellopress/wp_filemanager.php. Para comprobar si había sido algo puntual, se revisaron directamente los logs de Nginx buscando esta ruta.

```
ubuntu@honeypot-aegis:~$ sudo zgrep -h 'GET /wp-content/plugins/hellopress/wp_filemanager.php' \
/var/log/nginx/access.log
20.220.10.235 - - [25/Aug/2026:04:26:38 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
20.151.129.194 - - [24/Aug/2026:11:19:32 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
172.182.200.96 - - [24/Aug/2026:12:53:42 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
172.182.200.96 - - [24/Aug/2026:12:53:56 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
20.104.104.62 - - [24/Aug/2026:17:13:18 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
20.151.9.151 - - [24/Aug/2026:21:57:20 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
68.221.130.146 - - [17/Aug/2026:04:52:30 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
20.118.172.148 - - [17/Aug/2026:05:17:03 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
20.226.112.14 - - [17/Aug/2026:15:29:10 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
40.74.65.169 - - [17/Aug/2026:17:38:45 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
20.169.136.165 - - [17/Aug/2026:18:26:47 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
20.104.100.201 - - [16/Aug/2026:16:56:16 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
4.223.164.152 - - [16/Aug/2026:17:44:15 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
68.155.155.23 - - [16/Aug/2026:22:20:13 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
20.104.100.201 - - [15/Aug/2026:01:51:53 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
51.116.180.165 - - [15/Aug/2026:03:04:49 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
20.163.43.14 - - [15/Aug/2026:19:53:55 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
20.65.105.233 - - [15/Aug/2026:21:50:51 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
74.249.206.207 - - [14/Aug/2026:17:23:06 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
20.163.43.14 - - [14/Aug/2026:20:03:34 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
4.223.165.84 - - [12/Aug/2026:08:30:41 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
4.232.93.83 - - [12/Aug/2026:12:13:39 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
4.225.166.176 - - [12/Aug/2026:19:01:31 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 404 162 "-" "-"
ubuntu@honeypot-aegis:~$ sudo zgrep -h '20.220.10.235' /var/log/nginx/access.log
```

El resultado mostró algo interesante: varias IP diferentes habían buscado exactamente el mismo archivo en distintos momentos.

Esto nos decía que no se trataba de una IP repitiendo peticiones, sino de diferentes sistemas buscando el mismo recurso.

3.7.3 Investigación de la ruta detectada

Después de comprobar que diferentes IP estaban buscando el mismo archivo, se investigó qué podía significar esta ruta.

La dirección era una ubicación típica de plugins de WordPress, al investigar se vio que wp_filemanager.php puede estar relacionado con la búsqueda de una webshell, un archivo que puede encontrarse en servidores que han sido comprometidos y que permite interactuar con ellos a través de una interfaz web, los escáneres no parecían buscar si había una instalación de WordPress, sino si existía un archivo muy concreto que pudiera darles algún tipo de acceso.

Para comprobarlo mejor se revisó la actividad completa de una de las IP detectadas. En pocos segundos realizó numerosas peticiones buscando distintos archivos y rutas relacionadas con WordPress, lo que indicaba que probablemente se trataba de un proceso automatizado.

3.7.4 Creación de la falsa webshell

Una vez identificada la ruta, se decidió crear un recurso falso que simulase el archivo que estaban buscando los escáneres.

Para hacerlo de forma segura, no se instaló WordPress ni ninguna webshell real.

Simplemente se creó la misma estructura de carpetas y un archivo que imita un pequeño gestor de archivos

```
ubuntu@honeypot-aegis:~$ sudo mkdir -p /var/www/html/wp-content/plugins/hellopress
ubuntu@honeypot-aegis:~$ sudo nano /var/www/html/wp-content/plugins/hellopress/wp_filemanager.php
ubuntu@honeypot-aegis:~$ sudo chown -R root:root /var/www/html/wp-content
ubuntu@honeypot-aegis:~$ sudo chmod -R 755 /var/www/html/wp-content
ubuntu@honeypot-aegis:~$
```

```
GNU nano 7.2 /var/www/html/wp-content/plugins/hellopress/wp_filemanager.php
<!DOCTYPE html>
<html>
<head><title>File Manager</title></head>
<body>
<h3>File Manager</h3>
<form method="post">
<input type="text" name="path" value="/">
<input type="submit" value="Open">
</form>
</body>
</html>
```

La página muestra un título File Manager, un campo para introducir una ruta y un botón Open, pero no tiene ninguna funcionalidad real. No ejecuta PHP ni permite acceder, modificar o subir archivos al servidor.



La idea es que los escáneres que buscaban esta ruta recibían un 404 a partir de ahora encontrarán un recurso que existe, y podremos observar si después de recibir un 200 OK continúan realizando nuevas peticiones.

Para comprobar que el señuelo funcionaba se realizó una petición

```
mtk@mtk-IdeaPad-3-15IIL6:~$ curl -i https://aegisbank.fin-tech.com/wp-content/plugins/hellopress/wp_filemanager.php
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Date: Tue, 25 Aug 2026 09:22:22 GMT
Content-Type: application/octet-stream
Content-Length: 215
Last-Modified: Tue, 25 Aug 2026 09:08:37 GMT
Connection: keep-alive
```

4. Integración y detección con Wazuh

Una vez creados los señuelos, el siguiente paso fue configurar Wazuh para detectar la actividad que se produjera sobre ellos.

El agente instalado en el honeypot recoge los registros generados por Nginx y por los diferentes servicios y los envía al servidor Wazuh. Allí los eventos se analizan mediante reglas y se muestran en el Dashboard, así podemos identificar fácilmente cuándo alguien accede a uno de los señuelos y consultar información como la IP de origen, la ruta solicitada y el momento en que se produjo el acceso.

4.1 Monitorización de los logs de Nginx

Para detectar la actividad web se configuró Wazuh para monitorizar el registro de accesos de Nginx `/var/log/nginx/access.log`.

Este archivo registra las peticiones que recibe la web y permite a Wazuh analizar los accesos a los diferentes señuelos.

```

</ossec_config>
<ossec_config>
  <localfile>
    <log_format>journald</log_format>
    <location>journald</location>
  </localfile>

  <localfile>
    <log_format>apache</log_format>
    <location>/var/log/nginx/access.log</location>
  </localfile>

  <localfile>
    <log_format>apache</log_format>
    <location>/var/log/nginx/error.log</location>
  </localfile>

  <localfile>

```

4.2 Reglas personalizadas de detección

Ya cuando Wazuh recibía correctamente los logs del honeypot, se crearon varias reglas personalizadas para detectar los accesos a los señuelos.

La idea era que estas peticiones no quedaran mezcladas con el resto de actividad web, sino que pudieran identificarse fácilmente cuando alguien interactuara con alguno de los señuelos.

Las reglas creadas permiten detectar:

- el acceso al archivo falso /.env;
- el acceso a la ruta oculta del panel de administración;
- el acceso a la falsa webshell wp_filemanager.php.

Cada una dispone de su propio identificador y nivel de alerta, lo que permite localizarlas después desde Wazuh y diferenciarlas del resto de eventos recibidos.

```

user@wazuhcenter: $ sudo cat /var/ossec/etc/rules/deception_rules.xml
[sudo] password for user:
<group name="deception,honeypot,web,">

  <rule id="31108" level="0" overwrite="yes" noalert="1">
    <if_sid>31100</if_sid>
    <id>^2|^3</id>
    <compiled_rule>is_simple_http_request</compiled_rule>
    <description>Ignored URLs (simple queries).</description>
  </rule>

  <rule id="100200" level="7">
    <if_sid>31108</if_sid>
    <url type="pcre2">^/\.env$</url>
    <description>Deception: acceso al archivo .env señuelo desde $(srcip)</description>
    <group>deception_env,reconnaissance,</group>
  </rule>

  <rule id="100210" level="8">
    <if_sid>31108</if_sid>
    <url type="pcre2">^/wp-content/plugins/hellopress/wp_filemanager\.php$</url>
    <description>Deception: acceso al señuelo de webshell HelloPress desde $(srcip)</description>
    <group>deception_webshell,webshell_probe,suspicious_access,</group>
  </rule>
</group>

```


4.3 Prueba de las reglas

Una vez creadas las reglas, se hicieron varias pruebas con wazuh-logtest para comprobar que funcionaban.

Esta herramienta permite introducir una petición de prueba y ver qué hace Wazuh con ella, sin tener que esperar a que alguien acceda realmente al señuelo.

Se probaron las diferentes rutas y se comprobó que Wazuh reconocía cada una y generaba la alerta correspondiente.

Por ejemplo, al simular un acceso a la falsa webshell wp_filemanager.php, se activó correctamente la regla 100210, con nivel de alerta 8.

```
user@wazuhcenter:~$ sudo /var/ossec/bin/wazuh-logtest
Starting wazuh-logtest v4.14.7
Type one log per line

79.152.205.50 - - [26/Aug/2026:08:00:00 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 200 1234 "-" "Mozilla/5.0"

**Phase 1: Completed pre-decoding.
  full event: '79.152.205.50 - - [26/Aug/2026:08:00:00 +0000] "GET /wp-content/plugins/hellopress/wp_filemanager.php HTTP/1.1" 200 1234 "-" "Mozilla/5.0"'

**Phase 2: Completed decoding.
  name: 'web-accesslog'
  id: '200'
  protocol: 'GET'
  srcip: '79.152.205.50'
  url: '/wp-content/plugins/hellopress/wp_filemanager.php'

**Phase 3: Completed filtering (rules).
  id: '100210'
  level: '8'
  description: 'Deception: acceso al señuelo de webshell HelloPress desde 79.152.205.50'
  groups: ['deception', 'honeypot', 'web', 'deception_webshell', 'webshell_probe', 'suspicious_access']
  firetimes: '1'
  mail: 'False'

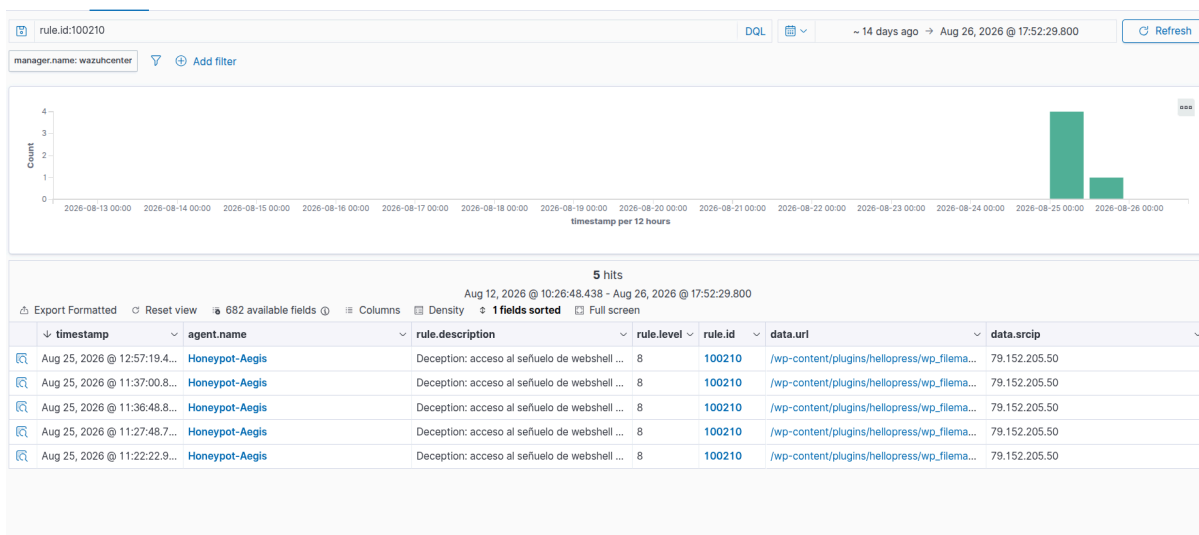
**Alert to be generated.
```

4.4 Comprobación de los señuelos en Wazuh

Una vez comprobadas las reglas con wazuh-logtest, se realizaron accesos controlados a los señuelos desde un equipo externo para verificar el funcionamiento.

Las peticiones llegaron primero a Nginx, quedaron registradas en sus logs y fueron recogidas por el agente de Wazuh instalado en Honeypot-Aegis.

Después, desde Threat Hunting, se comprobó que los accesos aparecían como alertas y que las reglas personalizadas se estaban aplicando bien.



5. Análisis de la actividad detectada

Una vez configurados los señuelos y las reglas de Wazuh, se dejó el honeypot expuesto a Internet para observar qué ocurría.

Durante este tiempo se recibieron numerosas conexiones y peticiones desde diferentes direcciones IP. Gran parte de ellas parecían proceder de sistemas automatizados que buscaban archivos, directorios o servicios concretos.

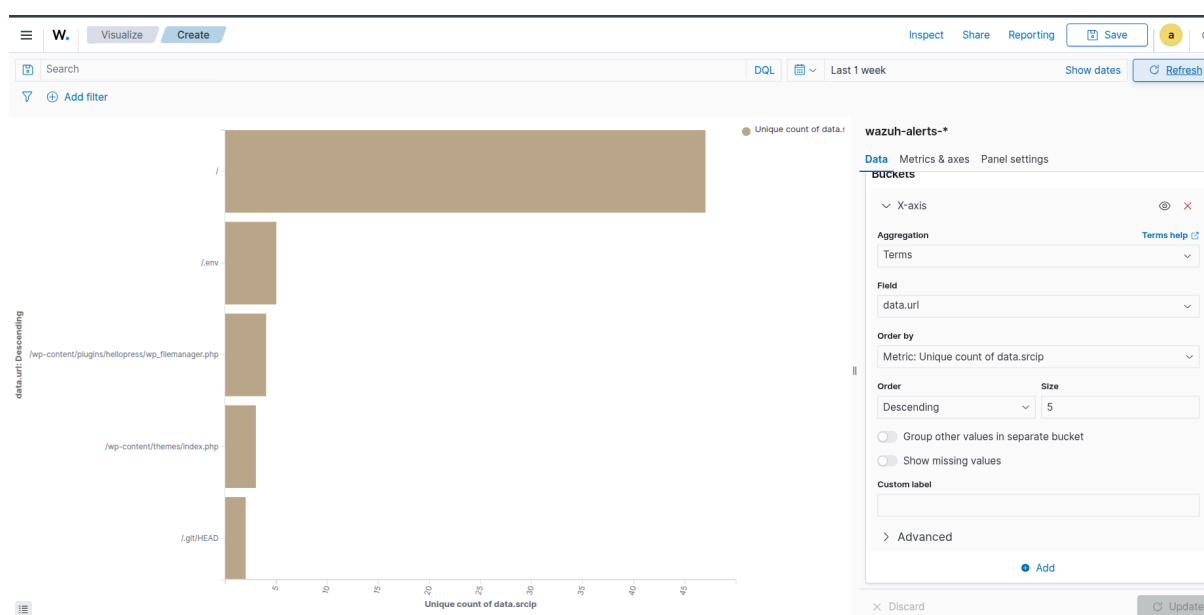
Toda esta información quedó centralizada en Wazuh, así que permitió revisar con más facilidad qué estaban buscando y cómo se comportaban algunas de las IP que llegaban al honeypot.

5.1 Análisis de las rutas más buscadas

Para ver qué recursos eran buscados con más frecuencia, se creó una visualización en Wazuh agrupando las peticiones por data.url.

En lugar de contar todas las peticiones, se tuvo en cuenta el número de IP diferentes que había accedido a cada ruta. Así se evita que una sola IP realizando muchas solicitudes tenga demasiado peso en los resultados.

Se observaron accesos habituales a /, pero también búsquedas mucho más interesantes, como archivos .env, rutas relacionadas con WordPress y archivos que podían estar asociados a webshells.



5.2 Análisis de una IP con alta actividad

Durante el análisis se encontraron algunas IP que realizaban muchas peticiones en muy poco tiempo.

Un ejemplo fue la dirección 20.220.10.235. Al revisar su actividad en Wazuh se pudo ver cómo iba probando diferentes archivos y rutas, muchas de ellas relacionadas con WordPress y archivos PHP.

Las peticiones se realizaban prácticamente una detrás de otra, lo que hace pensar que se trataba de un escaneo automatizado y no de una persona navegando manualmente por la web.

Además, este comportamiento se repitió en diferentes momentos: después de realizar una serie de peticiones, la actividad se detenía y volvía a aparecer unas horas más tarde.

270 hits									
Jul 25, 2008 @ 10:50:14:569 - Aug 25, 2008 @ 10:50:07569									
Export Formatted	Reset view	692 available fields	Columns	Display	Fields sorted	Full access			
Time range	Agent name	Rule description	Rule level	Rule ID	Data type	Count	Time range	Fields sorted	Data type
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	net/7p	5	Aug 25, 2008 @ 10:56:35.0	net/7p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	net/4p	5	Aug 25, 2008 @ 10:56:35.0	net/4p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100.php	5	Aug 25, 2008 @ 10:56:35.0	/100.php	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	net/100/indirect.php	5	Aug 25, 2008 @ 10:56:35.0	net/100/indirect.php	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Multiple web server 400 error codes from...	10	31051	/11p/4p	5	Aug 25, 2008 @ 10:56:35.0	/11p/4p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p/100p/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p/100p/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p/100p/100p/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p/100p/100p/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p/100p/100p/100p/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p/100p/100p/100p/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p/100p/100p/100p/100p/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p/100p/100p/100p/100p/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p/100p/100p/100p/100p/100p/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p/100p/100p/100p/100p/100p/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p/100p/100p/100p/100p/100p/100p/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p/100p/100p/100p/100p/100p/100p/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p/100p/100p/100p/100p/100p/100p/100p/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p/100p/100p/100p/100p/100p/100p/100p/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p	5	Aug 25, 2008 @ 10:56:35.0	/100p	5
Aug 25, 2008 @ 10:56:35.0	Network-Angel	Web server 400 error codes	5	31001	/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100p/100				

Al revisar los logs de Nginx se observó que varias IP seguían buscando la ruta del tercer señuelo `/wp-content/plugins/hellopress/wp_filemanager.php`. Lo curioso era que estas peticiones seguían recibiendo un 404, aunque el señuelo ya estaba creado.

Al revisar la configuración se descubrió el motivo: muchos escáneres no utilizaban el dominio, sino que accedían directamente a la IP pública del honeypot mediante HTTP. El señuelo, en cambio, estaba preparado para responder a través del dominio y HTTPS. Así que se ajustó la configuración de Nginx para que la ruta también estuviera disponible cuando la petición llegara directamente por HTTP.

5.4 Resultados observados

Durante el tiempo que el honeypot ha estado expuesto a Internet se ha ido registrando actividad de forma prácticamente continua, gran parte de ella procedente de escaneos y bots automatizados.

En la parte web se han observado búsquedas de archivos sensibles, rutas relacionadas con WordPress y posibles webshells. Algunas de estas peticiones han llegado hasta los señuelos preparados y han generado sus correspondientes alertas en Wazuh.

Durante algunos periodos, el agente Wazuh perdió la conexión con el Manager instalado en local. Mientras esto ocurría, Nginx continuó registrando normalmente las peticiones recibidas, pero esos eventos no aparecieron como alertas en Wazuh.

```
ubuntu@honeypot-aegle:~$ sudo zgrep -h '31.59.18.21' /var/log/nginx/access.log*
31.59.18.21 - - [26/Aug/2026:05:28:23 +0000] "GET /.env HTTP/1.1" 200 160 "-" Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/116.0.5845.140 Safari/537.36"
31.59.18.21 - - [26/Aug/2026:05:28:25 +0000] "POST / HTTP/1.1" 405 568 "-" Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/116.0.5845.140 Safari/537.36"
ubuntu@honeypot-aegle:~$
```

La IP 31.59.18.21 accedió al archivo .env y recibió un 200 OK. Solo dos segundos después realizó una petición POST, que fue rechazada con un 405.

Aunque no se puede confirmar que ambas acciones estén relacionadas, resulta interesante que la misma IP continuara interactuando con el servidor después de encontrar el señuelo.

Cowrie también ha seguido recibiendo conexiones. En una de las sesiones, por ejemplo, un sistema consiguió entrar en el entorno simulado utilizando las credenciales root:123, y después el comando `ps aux | head -10`

The image displays two side-by-side screenshots of the Wazuh dashboard's 'Document Details' view. Both screenshots show a document with a title 'JSON' and a table of data. The left screenshot shows a document with a title 'JSON' and a table of data including 'data.url', 'data.method', and 'data.status'. The right screenshot shows a document with a title 'JSON' and a table of data including 'data.url', 'data.method', and 'data.status'.

5.5 Detección de actividad compatible con AndroXGh0st

Durante la monitorización apareció un patrón que llamó especialmente la atención. Desde diferentes direcciones IP se repetía prácticamente la misma secuencia, primero se solicitaba el archivo .env y, apenas un segundo después, se enviaba una petición POST contra la página principal.

Gracias al registro de las peticiones POST se pudo ver también el contenido enviado que era 0x%5B%5D=androxgh0st, que decodificado era 0x[]=androxgh0st.

```
ubuntu@honeypot-aegis:~$ sudo zgrep -h '82.21.245.36' /var/log/nginx/access.log*
82.21.245.36 - - [27/Aug/2026:13:09:41 +0000] "GET /.env HTTP/1.1" 404 196 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/116.0.5845.140 Safari/537.36"
82.21.245.36 - - [27/Aug/2026:13:09:42 +0000] "POST / HTTP/1.1" 200 3 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/116.0.5845.140 Safari/537.36"
```

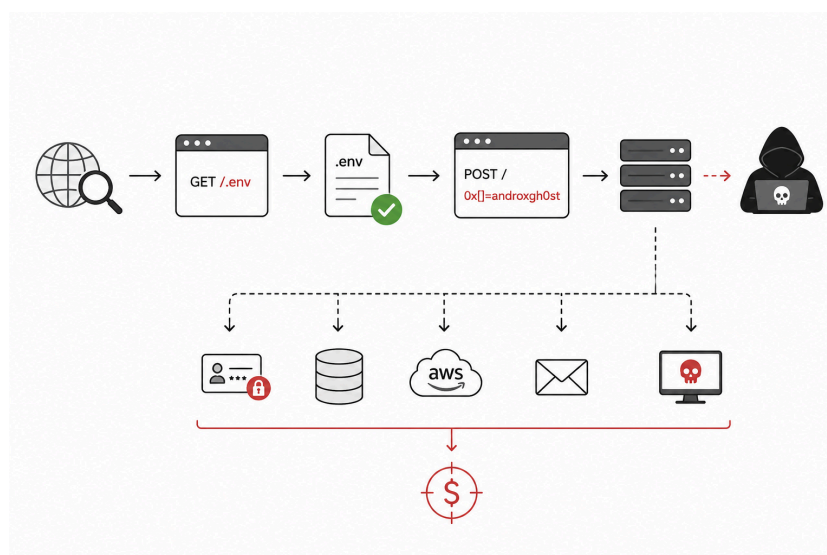
El mismo comportamiento apareció desde al menos dos IP diferentes, siguiendo la misma secuencia y utilizando el mismo contenido.

Aug 27, 2026 @ 15:09:43.735	HoneyPot-Aegis	Deception: POST capturado desde 82.21.245.36	10	100230	-	82.21.245.36	-	-
Aug 27, 2026 @ 14:05:31.308	HoneyPot-Aegis	Deception: POST capturado desde 82.23.203.135	10	100230	-	82.23.203.135	-	-
Aug 27, 2026 @ 14:05:29.839	HoneyPot-Aegis	Deception: acceso al archivo .env señuelo de...	7	100200	/.env	82.23.203.135	-	GET
Aug 27, 2026 @ 13:28:19.817	HoneyPot-Aegis	Deception: POST capturado desde 2.144.22.46	10	100230	-	2.144.22.46	-	-
Aug 27, 2026 @ 13:28:19.817	HoneyPot-Aegis	Deception: POST capturado desde 2.144.22.46	10	100230	-	2.144.22.46	-	-

Aug 27, 2026 @ 15:09:43.735	HoneyPot-Aegis	Deception: POST capturado	data.scrip 82.23.283.135
Aug 27, 2026 @ 14:05:31.308	HoneyPot-Aegis	Deception: POST capturado	data.user_agent Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/116.0.5845.140 Safari/537.36
Aug 27, 2026 @ 14:05:29.839	HoneyPot-Aegis	Deception: acceso al archivo .env	decoder.name aegis-post
Aug 27, 2026 @ 13:28:19.817	HoneyPot-Aegis	Deception: POST capturado	full_log 2026-08-27T12:05:30.267258+00:00 src_ip=82.23.283.135 content_type='application/x-www-form-urlencoded' content_length=20 user_agent='Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/116.0.5845.140 Safari/537.36' body='0x%5B%5D=androxgh0st'
Aug 27, 2026 @ 11:40:32.941	HoneyPot-Aegis	Deception: POST capturado	id 1787832331.565200
Aug 27, 2026 @ 11:18:21.352	HoneyPot-Aegis	Deception: acceso al archivo .env	input.type text

Al investigar este patrón se encontraron indicadores publicados por CISA relacionados con AndroXGh0st, una amenaza que, entre otras acciones, busca archivos .env expuestos para localizar posibles credenciales y claves almacenadas en ellos.

<https://www.cisa.gov/news-events/cybersecurity-advisories/aa24-016a>



Con estos datos no se puede asegurar que las IP pertenezcan realmente a AndroxGh0st, pero el comportamiento observado coincide con el que se ha documentado para esta amenaza.

Este caso fue especialmente interesante porque el señuelo .env permitió ver qué hacía el sistema después de encontrar el archivo y relacionar ese comportamiento con una amenaza conocida.

5.6 Limitaciones observadas

Durante las pruebas hubo algunos momentos en los que el agente Wazuh perdió la conexión con el servidor local. Nginx siguió registrando las peticiones, pero algunos eventos no llegaron al Dashboard.

Esto ocurrió, por ejemplo, con el acceso de la IP 31.59.18.21 al archivo .env, que quedó registrado en Nginx pero no en Wazuh.

Esto demuestra la importancia de conservar también los logs locales y no depender únicamente del SIEM para revisar la actividad.

```
ubuntu@honeypot-aegls:~$ sudo grep "2026/08/26 05:" /var/ossec/logs/ossec.log
2026/08/26 05:01:02 wazuh-agentd: ERROR: (1200): Unable to connect to enrollment service at '[10.10.10.2]:1515'
2026/08/26 05:01:12 wazuh-agentd: WARNING: (4101): Waiting for server reply (not started). Tried: '10.10.10.2'. Ensure that the manager version is 'v4.14.7' or higher.
2026/08/26 05:01:12 wazuh-agentd: WARNING: Unable to connect to any server.
2026/08/26 05:01:12 wazuh-agentd: INFO: Trying to connect to server ([10.10.10.2]:1514/tcp).
2026/08/26 05:03:25 wazuh-agentd: ERROR: (1216): Unable to connect to '[10.10.10.2]:1514/tcp': 'Transport endpoint is not connected'.
2026/08/26 05:03:35 wazuh-agentd: INFO: Trying to connect to server ([10.10.10.2]:1514/tcp).
2026/08/26 05:05:48 wazuh-agentd: ERROR: (1216): Unable to connect to '[10.10.10.2]:1514/tcp': 'Transport endpoint is not connected'.
2026/08/26 05:05:58 wazuh-agentd: INFO: Trying to connect to server ([10.10.10.2]:1514/tcp).
2026/08/26 05:08:12 wazuh-agentd: ERROR: (1216): Unable to connect to '[10.10.10.2]:1514/tcp': 'Transport endpoint is not connected'.
2026/08/26 05:08:22 wazuh-agentd: INFO: Trying to connect to server ([10.10.10.2]:1514/tcp).
2026/08/26 05:10:35 wazuh-agentd: ERROR: (1216): Unable to connect to '[10.10.10.2]:1514/tcp': 'Transport endpoint is not connected'.
2026/08/26 05:10:45 wazuh-agentd: INFO: Trying to connect to server ([10.10.10.2]:1514/tcp).
2026/08/26 05:12:59 wazuh-agentd: ERROR: (1216): Unable to connect to '[10.10.10.2]:1514/tcp': 'Transport endpoint is not connected'.
2026/08/26 05:12:59 wazuh-agentd: INFO: Requesting a key from server: 10.10.10.2
2026/08/26 05:15:14 wazuh-agentd: ERROR: (1200): Unable to connect to enrollment service at '[10.10.10.2]:1515'
2026/08/26 05:15:24 wazuh-agentd: WARNING: (4101): Waiting for server reply (not started). Tried: '10.10.10.2'. Ensure that the manager version is 'v4.14.7' or higher.
2026/08/26 05:15:24 wazuh-agentd: WARNING: Unable to connect to any server.
2026/08/26 05:15:24 wazuh-agentd: INFO: Trying to connect to server ([10.10.10.2]:1514/tcp).
2026/08/26 05:17:37 wazuh-agentd: ERROR: (1216): Unable to connect to '[10.10.10.2]:1514/tcp': 'Transport endpoint is not connected'.
2026/08/26 05:17:47 wazuh-agentd: INFO: Trying to connect to server ([10.10.10.2]:1514/tcp).
2026/08/26 05:20:00 wazuh-agentd: ERROR: (1216): Unable to connect to '[10.10.10.2]:1514/tcp': 'Transport endpoint is not connected'.
```

6. Conclusiones

Este proyecto ha permitido ampliar el honeypot original añadiendo nuevos señuelos y, sobre todo, comprobar cómo reaccionan sistemas reales cuando encuentran un servidor expuesto a Internet.

Durante el tiempo de monitorización se han observado principalmente escaneos automatizados, búsquedas de archivos y rutas concretas, además de conexiones SSH e interacción con Cowrie.

Wazuh ha permitido reunir y analizar toda esta información de una forma mucho más visual. También se ha comprobado la importancia de conservar los logs locales, ya que durante algunos problemas de conexión con el Manager estos permitieron recuperar actividad que no había llegado al Dashboard.

Una de las partes más interesantes ha sido poder adaptar los señuelos a partir de lo que realmente se estaba observando. El tercer señuelo, por ejemplo, surgió después de detectar que diferentes IP buscaban repetidamente una misma ruta.

Como trabajo futuro, sería interesante mantener el honeypot activo durante más tiempo para seguir recogiendo actividad y descubrir nuevos patrones, intentos de explotación o muestras de malware. Esto permitiría continuar mejorando los señuelos según el comportamiento que se vaya observando.